Chapter 1

Advanced C Programming Techniques

ENEL712 Embedded Systems Design — Lecture 11

Summary

Advanced C techniques for embedded systems: switch-case state machines, advanced datatypes (arrays, strings, pointers, structures, function pointers), standard library helpers, and variable scope.

Topics Covered

- Switch-case state machines
- Advanced datatypes
- Pointer concepts and arithmetic
- Standard C library functions
- Variable access and scope
- Embedded risks with strings and arrays

1.1 Switch-Case State Machines

- FSM is the standard pattern for discrete processes (multi-stage flows, comms, UI, simple control).
- In C, naturally implemented using switch-case.
- Design with pen and paper or a graphics tool before coding.
- Always end each case with break to avoid case fall-through.

```
typedef enum { IDLE, MEASURE, SEND } state_t;
state_t state = IDLE;

for(;;) {
  switch(state) {
    case IDLE:    if (timer_flag) state = MEASURE; break;
```

```
    case MEASURE: sample = read_sensor(); state = SEND; break;
    case SEND:    uart_send(sample); state = IDLE; break;
  }
}
```

1.2 Advanced Datatypes

Datatype	Description	Key features
Arrays	Data stored in consecutive memory locations	Pointer arithmetic; same-type elements; usually 1D/2D
Strings	Char arrays terminated with '\{}0'	Not a native type; ASCII; required for std-lib string funcs
Pointers	Variables holding a memory address	Access without copying; allow funcs to modify args
Structures	Group multiple variables of different types under one name	. for direct access, -> for pointers
Function pointers	Pointers that point to functions	Useful for callbacks and search algorithms

1.3 Pointer Concepts and Arithmetic

- Reference (&): returns the address of a variable.
- Dereference (*): accesses the value at an address.
- Pointers are integer addresses, so arithmetic is allowed (e.g., incrementing an array pointer moves to the next element).
- Dangling pointers (uninitialised) are bad practice.
- In embedded C, pointers access hardware registers (peripherals at fixed addresses), pass large structs efficiently, and implement callbacks/jump tables.

1.4 Standard C Library Functions

1.4.1 <stdio.h>

- printf — formatted string to stdout.
- sprintf — like printf but writes to a char array (very useful in embedded).
- sscanf — read formatted data from a string.

1.4.2 <stdlib.h>

- atoi, atof, atol — quickly convert strings to int/float/long (no error reporting).

1.4.3 <string.h>

- memcpy — copy N bytes between memory locations.
- memset — set an entire array to a specified value (e.g., zero-init).
- strcpy — copy null-terminated character arrays.

1.5 Variable Access and Scope

- extern: share a global variable across files — define in one .c, declare extern in a header.
- Local static: a local variable that retains its value between calls.
- Global static: restricts a global variable or function to its source file (internal linkage).
- Risk: static-local variables make a function non re-entrant — unsafe in interrupts/threads.

1.6 Embedded Risks with Strings and Arrays

- Buffer overflow: writing past an allocated buffer corrupts adjacent data and risks security.
- Missing null terminator: standard library reads past the intended end.
- printf/sprintf are large in code and RAM — use carefully on resource-constrained devices.

```
typedef struct {
  int16_t  temp_c_x100;
  uint32_t timestamp_ms;
  uint8_t  status;
} temp_record_t;
```

1.7 Use Cases & Applications

- Function pointers as callbacks: a generic timer driver can take 'on tick' as a function pointer so different modules wire up their own logic without modifying the driver.
- Structs as records: a single struct passed by pointer keeps a complex measurement (timestamp + temp + humidity + ...) coherent across function calls.
- Pointer-based parsing: an incoming UART buffer is parsed by walking a pointer through it, picking out fields without copying.
- FSMs implemented with switch-case scale to dozens of states — when they reach hundreds, prefer state-tables of function pointers.

1.8 Code Snippets

1.8.1 Function pointer / callback

```
typedef void (*tick_handler_t)(void);
static tick_handler_t on_tick;

void timer_set_handler(tick_handler_t fn) {
    on_tick = fn;
}

ISR(TIMER1_OVF_vect) {
    if (on_tick) on_tick();
}
```

1.8.2 Static local for state

```
uint16_t debounce(uint8_t pin) {
    static uint16_t shift = 0;        // retains value between calls
    shift = (shift << 1) | pin;
    return shift == 0xFFFF;           // 16 stable highs
}
```

1.8.3 sprintf-into-buffer for UART output

```
char buf[40];
int n = snprintf(buf, sizeof(buf),
                 "T=%d.%02d C, RH=%d %%\r\n",
                 t / 100, t % 100, rh);
uart_write(buf, n);
```

1.9 Common Pitfalls

- Buffer overflow: snprintf is safer than sprintf because it takes the buffer length.
- Forgetting the null terminator when manually building strings — string functions read past the intended end.
- Using an interrupt-shared variable without volatile causes the compiler to cache it in a register and miss updates from the ISR.
- Calling printf inside an ISR can blow the stack; ISRs should be short and only set flags or push to a queue.

Key Definitions

FSM
Finite State Machine; pattern for multi-stage processes implemented with switch-case in C.

Dangling Pointer
Pointer that has been declared but does not point to a valid memory address.

extern
Declares a variable/function defined in another translation unit.

static (local)
Local variable with static storage; retains value between calls.

static (global/file scope)
Restricts a variable/function to its own source file (internal linkage).

Function Pointer
Variable storing the address of a function; used for callbacks and configurable strategies.

Pointer Arithmetic
Mathematical operations on pointers that move them by the size of the pointed-to type.

Null Terminator
'\{}0' character marking the end of a C string.

Callback
: Function pointer passed to another module so that module can 'call back' into yours when an event occurs.

Buffer Overflow
: Writing past the end of an allocated buffer; corrupts adjacent memory and is a classic security and reliability bug.

Re-entrancy
: Whether a function can be safely called recursively or from interrupts; static locals and shared globals break re-entrancy.

snprintf
: Length-bounded variant of sprintf that truncates rather than overrunning the buffer.

Sources

- Lecture 11.pdf